

3. Longest Substring Without Repeating Characters

Given a string *s*, find the length of the **longest substring** without duplicate characters.

Example 1:

Input: *s* = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: *s* = "bbbbbb"

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: *s* = "pwwkew"

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

The screenshot shows the LeetCode interface for the problem "3. Longest Substring Without Repeating Characters". The left panel contains the problem description, examples, and a hint. The right panel shows a Python solution and the test results.

Problem Description: Given a string *s*, find the length of the **longest substring** without duplicate characters.

Examples:

- Example 1:** Input: *s* = "abcabcbb", Output: 3. Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
- Example 2:** Input: *s* = "bbbbbb", Output: 1. Explanation: The answer is "b", with the length of 1.
- Example 3:** Input: *s* = "pwwkew", Output: 3. Explanation: The answer is "wke", with the length of 3. Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Python Solution:

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        l = 0
        longest = 0
        seen = set()

        for r in range(len(s)):
            while s[r] in seen:
                seen.remove(s[l])
                l = l + 1

            seen.add(s[r])
            w = r - l + 1
            longest = max(longest, w)

        return longest
```

Test Results: Accepted. Runtime: 0 ms. All test cases (Case 1, Case 2, Case 3) passed.

Input: *s* = "abcabcbb"